DAY 05 / 15 · PHASE 2 — ETHEREUM & SOLIDITY

# Ethereum & the EVM

Monday 19 October 2026 · Manish Khadka · github.com/Khadka1996/Learning-Blockchain/tree/main/day-05

## Learning objectives

1. Explain the two kinds of Ethereum account (EOA and contract) and what a nonce does.
2. Read EVM bytecode as a list of opcodes and trace what each one does to the stack.
3. Work out the gas of a transaction: the 21,000 base cost, cheap opcodes, expensive storage, cold vs warm access and refunds.
4. Find any Solidity variable in raw storage: packed slots, mappings at `keccak256(key, slot)`, arrays at `keccak256(slot) + i`.
5. Prove a model is right by testing it against the real thing (differential testing).

## The concept, step by step

### Step 1 — From a coin ledger to a world computer

Days 01–04 built a ledger that only tracks **who owns how many coins**. Ethereum keeps that ledger and also **runs programs** on it. Every node runs the same small computer, the **EVM** (Ethereum Virtual Machine), on the same inputs, and every node must get exactly the same result. If one node got a different answer, the network would split.

### Step 2 — Two kinds of account

| | Externally owned account (EOA) | Contract account |
|---|---|---|
| **Controlled by** | A private key (a person or a bot, Day 03) | Its own code |
| **Has code?** | No | Yes, fixed once deployed |
| **Has storage?** | No | Yes, $2^{256}$ slots of 32 bytes |
| **Balance + nonce** | Yes: nonce = transactions sent | Yes: nonce = contracts it created |
| **Starts a transaction?** | Yes | No, only reacts to calls |

The **nonce** is a counter. Each transaction must use the next number, so the same signed transaction can never be played twice. A contract's address is also worked out from its creator and nonce: `keccak256(rlp(sender, nonce))`. In my run the predicted and real address matched exactly.

### Step 3 — The EVM is a calculator with a stack

Contract code is a list of one-byte instructions called **opcodes** (`0x01` = `ADD`, `0x55` = `SSTORE`). Each opcode takes numbers off the top of a stack of 256-bit numbers and puts its answer back on top. Here is "store 2 + 3 in slot 0":




Figure 1 — The stack after each opcode. Top of the stack is highlighted. Total execution gas: 3+3+3+3+22,100.

### Step 4 — Gas: the price list that keeps the computer alive

Every opcode costs **gas**. You pay `gas used × gas price`. Gas does two jobs: endless loops stop because they run out of gas, and spam costs real money because thousands of nodes repeat your work. A plain ETH transfer always costs exactly **21,000** gas.

The biggest costs are in storage, because every node must keep that data for ever. The first time a transaction touches a slot it is **cold** (2,100 extra); after that it is **warm** (100), from EIP-2929.

| SSTORE write (cold slot) | Gas | Why |
|---|---|---|
| 0 → x (new value) | 22,100 | A new 32-byte value every node must keep |
| x → y (change) | 5,000 | The slot already exists |
| x → x (no change) | 2,200 | You only pay for the access |
| x → 0 (clear) | 5,000 − 4,800 refund | Freeing state is rewarded (EIP-3529: refund ≤ ⅕ of gas used) |

### Step 5 — Storage: a wall of lockers

A contract has $2^{256}$ lockers (slots) of 32 bytes, all zero until written. Solidity fills them in the order variables are declared and **packs** small values into one slot when they fit. Mappings and dynamic arrays cannot be stored in order (they can grow without limit), so their items are placed at `keccak256` addresses where they will never collide.

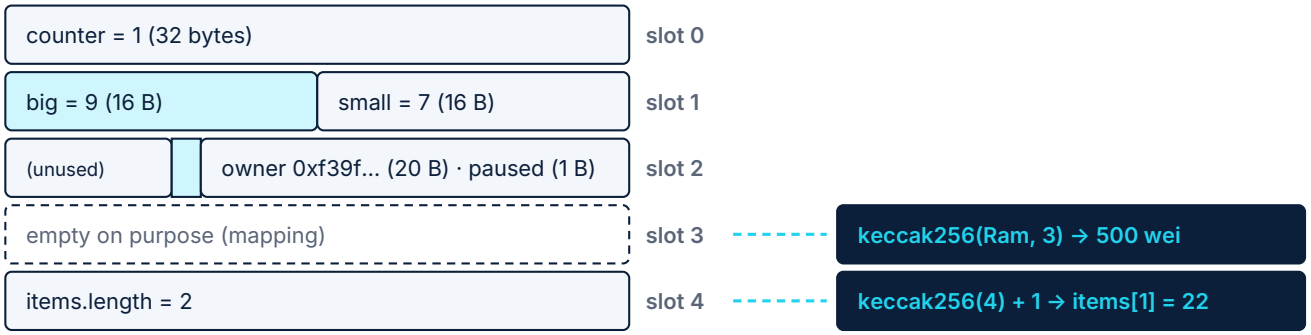



Figure 2 — Real storage of Layout.sol read with eth_getStorageAt. Packed values share a slot; mappings and arrays are scattered by keccak256.

## Key code walkthrough

### 1. The gas price list (src/evm.js)

```
// Mainnet gas constants. Storage is priced 1,000× higher than arithmetic.
zero: 0, base: 2, verylow: 3, low: 5, mid: 8, high: 10, jumpdest: 1,
coldSload: 2100, warmRead: 100, sset: 20000, sreset: 2900, clearRefund: 4800,
const MAX_GAS = 30_000_000;  // one block; also caps memory at ~4 MB (QA fix M1)
```

### 2. Cold vs warm inside SSTORE

```
// First touch of a slot in this transaction adds the cold surcharge, then it is warm.
if (!s.warm.has(key)) { s.warm.add(key); cost += G.coldSload; }
if (value === 0n) s.refund += G.clearRefund;     // clearing a slot earns a refund
// at the end: refund is capped at 1/5 of the gas used (EIP-3529)
const refund = Math.min(Math.max(s.refund, 0), Math.floor(raw / G.maxRefundQuotient));
```

### 3. Finding a variable in storage (src/storage.js)

```
// balances[key] when the mapping is declared at `slot`
function mappingSlot(keyType, key, slot) {
  return BigInt(keccak256(coder.encode([keyType, "uint256"], [key, slot])));
}
// items[i] when the array is declared at `slot`: keccak256(slot) + i
function arraySlot(slot, index) {
  const start = BigInt(keccak256(zeroPadValue(toBeHex(slot), 32)));
  return (start + BigInt(index)) % (1n << 256n);
}
// read a packed field: shift right by its offset, keep `size` bytes
function unpack(word, offset, size) {
  return (getBigInt(word) >> BigInt(8 * offset)) & ((1n << BigInt(8 * size)) - 1n);
}
```

### 4. The contract that shows the layout (contracts/Layout.sol)

```
uint256 public counter;               // slot 0
uint128 public small; uint128 public big;  // slot 1, packed: small low 16 B, big high 16 B
address public owner; bool public paused;  // slot 2, packed: 20 + 1 bytes
mapping(address => uint256) public balances; // slot 3 stays empty
uint256[] public items;              // slot 4 = length
```

## Results from my run

| What I measured | Result |
|---|---|
| Sita sends Ram 1 ETH | 21,000 gas × 1.875 gwei = fee 0.000039375 ETH; nonce now 1 |
| Predicted vs real contract address | 0xe7f1725E…0512 — **match** |
| Deploying Layout.sol | 736,081 gas, 2,952 bytes of code |
| `increment()` 0 → 1, then 1 → 2 | 43,567 gas, then 26,467 gas — **39% cheaper** the second time |
| "2 + 3, store, read, return" on my mini-EVM vs Hardhat | 43,230 vs 43,230 gas — **match**, returned 5 |
| Differential fuzz test | 150 random programs, 10,134 opcode steps in 300 transactions — every step's gas identical |

| | |
|---|---|
| Quality | 45 / 45 tests pass, lint clean, npm audit 0, Slither 0 high/medium/low |

Two surprises: the same function costs much less the second time (the first call *creates* a value, the second only *changes* it), and `SLOAD` right after `SSTORE` costs just 100, because the slot is already warm.

## QA & security findings

| ID | Severity | Found by | Problem in plain words | Fix |
|---|---|---|---|---|
| M1 | **Medium** | Manual | No upper gas limit: one MSTORE could make Node try to allocate 4 GB of memory. | gasLimit must be 21,000 – 30,000,000 (one block). |
| D1 | **Low** | Differential test | Running off the end of code: the real EVM records an implicit STOP, mine did not. | Record the implicit STOP. |
| L1 | **Low** | Manual | A long loop could keep millions of trace steps in memory. | Keep at most 100,000 steps, flag `traceTruncated`. |
| L2 | **Low** | Manual | Storage with Number keys (0 instead of 0n) silently read as zero. | Require `Map<bigint, bigint>`. |
| L3 | **Low** | Manual | Code starting with 0xEF was accepted though every chain refuses it (EIP-3541). | Refuse it up front with a clear error. |

Also attacked and found safe: re-entering `withdraw`, owner-only functions, overflow, replay across chains (EIP-155) and with a used nonce, jumps into PUSH data, and ETH forced in by `selfdestruct`.

## Common mistakes

- **Thinking gas price = gas.** Gas measures work; the price per unit changes with demand. Fee = gas used × gas price.
- **Forgetting the first write is expensive.** Setting a slot from 0 costs 22,100; designs that create many new slots get costly fast.
- **Reordering variables in an upgradeable contract.** The slot numbers move and old data is read as the wrong thing (Day 11).
- **Trusting a model you never compared with reality.** My unit tests passed; only the differential test caught D1.
- **Using `transfer` to send ETH.** Its 2,300-gas limit breaks smart-contract wallets; use `call` and check the result.

## Interview questions

**1. Why does the EVM need gas at all?**
Without it a single loop could freeze every node for ever. Gas makes every step cost something, so loops end and people pay for the work they ask the network to do.

**2. Where does Solidity store `balances[alice]` if the mapping is declared at slot 3?**
At `keccak256(abi.encode(alice, 3))`. Slot 3 itself stays empty.

**3. Why does the second `increment()` cost less than the first?**
The first changes the slot from zero (a new value, 20,000 + cold 2,100); the second only changes an existing value (2,900 + cold 2,100).

**4. What is the difference between an EOA and a contract account?**
An EOA is controlled by a private key and can start transactions; a contract account is controlled by its code, has storage, and only runs when called.

**5. How can you be sure a home-made EVM is correct?**
Differential testing: run the same random programs on it and on a reference EVM, and require identical gas at every step, the same storage and the same output.

## Exercises

| Level | Goal | Hint |
|---|---|---|
| Easy | Read `small` and `big` from packed slot 1. Expected: 7 and 9. | `unpack(word, 0, 16)` and `unpack(word, 16, 16)`; the first variable sits at the low end. |
| Medium | Write to slot 0, read it twice, return it; then clear it. | Look at the trace: first access 2,100 extra, then 100. Check the refund cap of ⅕. |
| Challenge | Add `MUL` or `AND` to the mini-EVM. | Unit-test stack, gas and 256-bit wrapping, then add a differential test against Hardhat. |

### Cheat sheet

**Costs to remember**
Plain transfer: 21,000
ADD / PUSH: 3 · MUL: 5
Cold storage access: +2,100 · warm: 100
New value 0 → x: 22,100 (cold)
Clear refund: 4,800, max ⅕ of gas used
Block limit: 30,000,000

**Where data lives**
Variables: slot 0, 1, 2 … in order
Small types: packed into one slot
`mapping` at slot p: `keccak256(key, p)`
`array` at slot p: length at p, item i at `keccak256(p) + i`
Contract address: `keccak256(rlp(sender, nonce))`

## References


1. Ethereum.org — Ethereum Virtual Machine: https://ethereum.org/en/developers/docs/evm/
2. Ethereum.org — Gas and fees: https://ethereum.org/en/developers/docs/gas/
3. Ethereum Yellow Paper (Appendix G fee schedule, Appendix H instruction set): https://ethereum.github.io/yellowpaper/paper.pdf
4. evm.codes — interactive opcode reference with gas costs: https://www.evm.codes/
5. Solidity docs — Layout of state variables in storage: https://docs.soliditylang.org/en/v0.8.24/internals/layout_in_storage.html
6. EIP-2929 — Gas cost increases for state access opcodes: https://eips.ethereum.org/EIPS/eip-2929
7. EIP-3529 — Reduction in refunds: https://eips.ethereum.org/EIPS/eip-3529
8. EIP-2200 — Structured definitions for net gas metering: https://eips.ethereum.org/EIPS/eip-2200
9. ethers.js v6 documentation: https://docs.ethers.org/v6/
10. Andreas M. Antonopoulos & Gavin Wood, *Mastering Ethereum*, ch. 13 "The EVM": https://github.com/ethereumbook/ethereumbook